

Análisis de ingeniería inversa sobre un sistema *Legacy*

Luis Augusto Rojas Guerra

12 de junio de 2026

Índice

1. Descripción general	3
1.1. Resumen de tablas	3
2. Inicialización de la base de datos	3
3. Definición de tablas	3
3.1. Tabla departamentos	3
3.2. Tabla profesores	4
3.3. Tabla cursos	4
3.4. Tabla estudiantes	5
3.5. Tabla inscripciones	5
3.6. Tabla aulas	6
3.7. Tabla horarios	6
3.8. Tabla notas	7
4. Relaciones entre tablas	8
5. Script completo de creación	8
6. Diagrama Entidad-Relación	10
7. Consultas de analisis y resultados	10
7.1. Consulta 1: Estudiantes inscritos en Algebra	10
7.2. Consulta 2: Cursos impartidos por Pedro Martinez	11
7.3. Consulta 3: Cantidad de estudiantes por curso	11
7.4. Consulta 4: Promedio de notas por curso	11
7.5. Consulta 5: Estudiantes con notas mayores a 8.0	12

7.6. Consulta 6: Cursos con mas de una inscripcion	12
7.7. Consulta 7: Profesores del departamento Ingenieria	12
7.8. Consulta 8: Cursos programados los lunes	13
7.9. Consulta 9: Aulas con capacidad mayor a 25	13
7.10. Consulta 10: Cursos con aula y horario	13
7.11. Consulta 11: Estudiante con la nota mas alta	14
7.12. Consulta 12: Cantidad de profesores por departamento	14
7.13. Consulta 13: Cursos sin estudiantes inscritos	14
7.14. Consulta 14: Estudiantes inscritos en mas de un curso	15
7.15. Consulta 15: Curso con menor promedio de notas	15

1. Descripción general

La base de datos **escuela** modela el sistema académico de una institución educativa. Contempla la gestión de departamentos, profesores, cursos, estudiantes, inscripciones, aulas, horarios y calificaciones, interrelacionados mediante claves foráneas que garantizan la integridad referencial.

1.1. Resumen de tablas

Tabla	Descripción	Columnas
departamentos	Áreas académicas de la institución	2
profesores	Docentes asignados a un departamento	5
cursos	Materias impartidas por un profesor	4
estudiantes	Alumnos registrados en el sistema	5
inscripciones	Relación entre estudiantes y cursos	4
aulas	Espacios físicos disponibles	3
horarios	Programación de cursos en aulas	6
notas	Calificaciones de cada inscripción	3

2. Inicialización de la base de datos

El bloque siguiente crea la base de datos si no existe y la selecciona como contexto activo para todas las sentencias SQL posteriores.

```
1 CREATE DATABASE IF NOT EXISTS escuela;  
2 USE escuela;
```

Listing 1: Creación y selección de la base de datos

3. Definición de tablas

3.1. Tabla departamentos

Almacena las áreas o facultades académicas a las que pertenecen los profesores. Es la entidad raíz de la jerarquía organizativa.

Columna	Tipo	Restricción	Descripción
id	INT	PK, AUTO_INCREMENT, NOT NULL	Identificador único
nombre	VARCHAR(100)	NOT NULL	Nombre del departame

```

1 CREATE TABLE departamentos (
2     id INT PRIMARY KEY AUTO_INCREMENT,
3     nombre VARCHAR(100) NOT NULL
4 );

```

Listing 2: SQL – departamentos

3.2. Tabla profesores

Registra a los docentes de la institución. Cada profesor pertenece a exactamente un departamento a través de la clave foránea `id_departamento`.

Columna	Tipo	Restricción	Descripción
<code>id</code>	INT	PK, AUTO_INCREMENT	Identificador único
<code>nombre</code>	VARCHAR(100)		Nombre del profesor
<code>apellido</code>	VARCHAR(100)		Apellido del profesor
<code>email</code>	VARCHAR(100)		Correo electrónico
<code>id_departamento</code>	INT	FK - departamentos	Departamento asignado

```

1 CREATE TABLE profesores (
2     id INT PRIMARY KEY AUTO_INCREMENT,
3     nombre VARCHAR(100),
4     apellido VARCHAR(100),
5     email VARCHAR(100),
6     id_departamento INT,
7     FOREIGN KEY (id_departamento) REFERENCES
8         departamentos(id)
9 );

```

Listing 3: SQL – profesores

3.3. Tabla cursos

Contiene las materias ofrecidas. Cada curso es impartido por un único profesor y tiene un valor en créditos académicos.

Columna	Tipo	Restricción	Descripción
<code>id</code>	INT	PK, AUTO_INCREMENT	Identificador único
<code>nombre</code>	VARCHAR(100)		Nombre del curso
<code>creditos</code>	INT		Valor en créditos
<code>id_profesor</code>	INT	FK - profesores	Profesor responsable

```

1 CREATE TABLE cursos (
2     id INT PRIMARY KEY AUTO_INCREMENT,
3     nombre VARCHAR(100),
4     creditos INT,
5     id_profesor INT,
6     FOREIGN KEY (id_profesor) REFERENCES profesores(id)
7 );

```

Listing 4: SQL – cursos

3.4. Tabla estudiantes

Registra a los alumnos matriculados. Incluye datos personales básicos y la fecha de nacimiento.

Columna	Tipo	Restricción	Descripción
id	INT	PK, AUTO_INCREMENT	Identificador único
nombre	VARCHAR(100)		Nombre del estudiante
apellido	VARCHAR(100)		Apellido del estudiante
email	VARCHAR(100)		Correo electrónico
fecha_nacimiento	DATE		Fecha de nacimiento

```

1 CREATE TABLE estudiantes (
2     id INT PRIMARY KEY AUTO_INCREMENT,
3     nombre VARCHAR(100),
4     apellido VARCHAR(100),
5     email VARCHAR(100),
6     fecha_nacimiento DATE
7 );

```

Listing 5: SQL – estudiantes

3.5. Tabla inscripciones

Tabla de unión que resuelve la relación muchos-a-muchos entre estudiantes y cursos. Registra también la fecha de inscripción.

Columna	Tipo	Restricción	Descripción
id	INT	PK, AUTO_INCREMENT	Identificador único
id_estudiante	INT	FK - estudiantes	Estudiante inscrito
id_curso	INT	FK - cursos	Curso al que se inscribe
fecha_inscripcion	DATE		Fecha de la inscripción

```

1 CREATE TABLE inscripciones (
2     id INT PRIMARY KEY AUTO_INCREMENT,
3     id_estudiante INT,
4     id_curso INT,
5     fecha_inscripcion DATE,
6     FOREIGN KEY (id_estudiante) REFERENCES estudiantes(
7         id),
8     FOREIGN KEY (id_curso) REFERENCES cursos(id)
9 );

```

Listing 6: SQL – inscripciones

3.6. Tabla aulas

Describe los espacios físicos donde se imparten las clases, con su capacidad máxima de estudiantes.

Columna	Tipo	Restricción	Descripción
id	INT	PK, AUTO_INCREMENT	Identificador único
nombre	VARCHAR(50)		Nombre o código del aula
capacidad	INT		Número máximo de alumnos

```

1 CREATE TABLE aulas (
2     id INT PRIMARY KEY AUTO_INCREMENT,
3     nombre VARCHAR(50),
4     capacidad INT
5 );

```

Listing 7: SQL – aulas

3.7. Tabla horarios

Programa los cursos en aulas específicas, indicando el día de la semana y el rango horario. Un mismo curso puede tener varios horarios.

Columna	Tipo	Restricción	Descripción
id	INT	PK, AUTO_INCREMENT	Identificador único
id_curso	INT	FK - cursos	Curso programado
id_aula	INT	FK - aulas	Aula asignada
dia_semana	VARCHAR(15)		Día de la semana
hora_inicio	TIME		Hora de inicio
hora_fin	TIME		Hora de término

```

1 CREATE TABLE horarios (
2     id INT PRIMARY KEY AUTO_INCREMENT,
3     id_curso INT,
4     id_aula INT,
5     dia_semana VARCHAR(15),
6     hora_inicio TIME,
7     hora_fin TIME,
8     FOREIGN KEY (id_curso) REFERENCES cursos(id),
9     FOREIGN KEY (id_aula) REFERENCES aulas(id)
10 );

```

Listing 8: SQL – horarios

3.8. Tabla notas

Almacena las calificaciones obtenidas por los estudiantes en cada inscripción. El tipo DECIMAL(4,2) admite valores hasta 99.99 con dos decimales.

Columna	Tipo	Restricción	Descripción
id	INT	PK, AUTO_INCREMENT	Identificador único
id_inscripcion	INT	FK - inscripciones	Inscripción evaluada
nota	DECIMAL(4,2)		Calificación obtenida

```

1 CREATE TABLE notas (
2     id INT PRIMARY KEY AUTO_INCREMENT,
3     id_inscripcion INT,
4     nota DECIMAL(4,2),
5     FOREIGN KEY (id_inscripcion) REFERENCES
6         inscripciones(id)
7 );

```

Listing 9: SQL – notas

4. Relaciones entre tablas

Tabla origen	Columna FK	Tabla destino
profesores	id_departamento	departamentos
cursos	id_profesor	profesores
inscripciones	id_estudiante	estudiantes
inscripciones	id_curso	cursos
horarios	id_curso	cursos
horarios	id_aula	aulas
notas	id_inscripcion	inscripciones

5. Script completo de creación

```
1  -- Crear Base de Datos
2  CREATE DATABASE IF NOT EXISTS escuela;
3  USE escuela;
4
5  -- Tabla de Departamentos
6  CREATE TABLE departamentos (
7      id INT PRIMARY KEY AUTO_INCREMENT,
8      nombre VARCHAR(100) NOT NULL
9  );
10
11 -- Tabla de Profesores
12 CREATE TABLE profesores (
13     id INT PRIMARY KEY AUTO_INCREMENT,
14     nombre VARCHAR(100),
15     apellido VARCHAR(100),
16     email VARCHAR(100),
17     id_departamento INT,
18     FOREIGN KEY (id_departamento) REFERENCES
        departamentos(id)
19 );
20
21 -- Tabla de Cursos
22 CREATE TABLE cursos (
23     id INT PRIMARY KEY AUTO_INCREMENT,
24     nombre VARCHAR(100),
25     creditos INT,
26     id_profesor INT,
27     FOREIGN KEY (id_profesor) REFERENCES profesores(id)
28 );
```



```

29
30 -- Tabla de Estudiantes
31 CREATE TABLE estudiantes (
32     id INT PRIMARY KEY AUTO_INCREMENT,
33     nombre VARCHAR(100),
34     apellido VARCHAR(100),
35     email VARCHAR(100),
36     fecha_nacimiento DATE
37 );
38
39 -- Tabla de Inscripciones
40 CREATE TABLE inscripciones (
41     id INT PRIMARY KEY AUTO_INCREMENT,
42     id_estudiante INT,
43     id_curso INT,
44     fecha_inscripcion DATE,
45     FOREIGN KEY (id_estudiante) REFERENCES estudiantes(
46         id),
47     FOREIGN KEY (id_curso) REFERENCES cursos(id)
48 );
49
50 -- Tabla de Aulas
51 CREATE TABLE aulas (
52     id INT PRIMARY KEY AUTO_INCREMENT,
53     nombre VARCHAR(50),
54     capacidad INT
55 );
56
57 -- Tabla de Horarios
58 CREATE TABLE horarios (
59     id INT PRIMARY KEY AUTO_INCREMENT,
60     id_curso INT,
61     id_aula INT,
62     dia_semana VARCHAR(15),
63     hora_inicio TIME,
64     hora_fin TIME,
65     FOREIGN KEY (id_curso) REFERENCES cursos(id),
66     FOREIGN KEY (id_aula) REFERENCES aulas(id)
67 );
68
69 -- Tabla de Notas
70 CREATE TABLE notas (
    id INT PRIMARY KEY AUTO_INCREMENT,

```

```

71     id_inscripcion INT,
72     nota DECIMAL(4,2),
73     FOREIGN KEY (id_inscripcion) REFERENCES
74         inscripciones(id)
    );

```

Script SQL completo - base de datos escuela

6. Diagrama Entidad-Relación

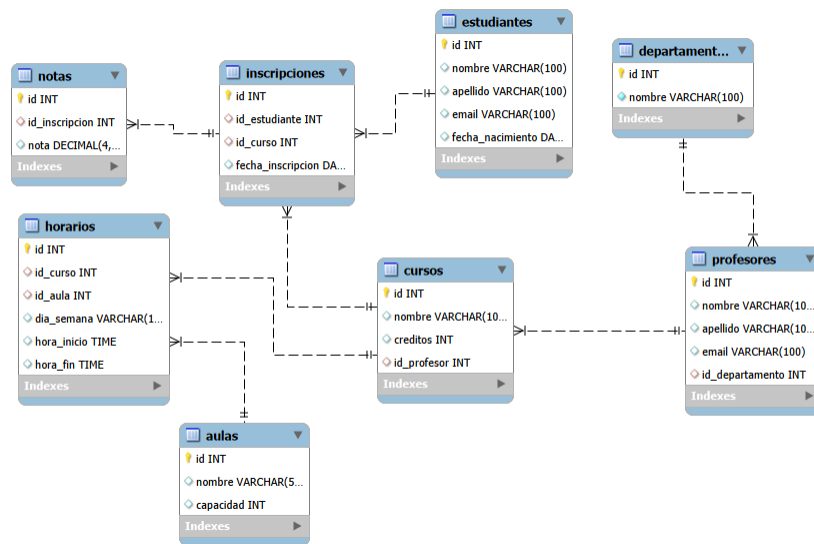


Figura 1: Diagrama EER - base de datos **escuela**

7. Consultas de análisis y resultados

7.1. Consulta 1: Estudiantes inscritos en Algebra

```

1 SELECT e.nombre, e.apellido, c.nombre AS curso
2 FROM estudiantes e
3 JOIN inscripciones i ON i.id_estudiante = e.id
4 JOIN cursos c ON c.id = i.id_curso
5 WHERE c.nombre = 'Álgebra';

```

Nombre	Apellido	Curso
Juan	Perez	Algebra
Maria	Gomez	Algebra

7.2. Consulta 2: Cursos impartidos por Pedro Martinez

```

1 SELECT c.nombre, p.nombre, p.apellido
2 FROM profesores p
3 LEFT JOIN cursos c ON c.id_profesor = p.id
4 WHERE p.nombre = 'Pedro'
5 AND p.apellido = 'Martínez';

```

Curso	Nombre	Apellido
Programacion	Pedro	Martinez

7.3. Consulta 3: Cantidad de estudiantes por curso

```

1 SELECT c.nombre,
2 COUNT(i.id_estudiante) AS total_estudiantes
3 FROM cursos c
4 LEFT JOIN inscripciones i ON i.id_curso = c.id
5 GROUP BY c.id, c.nombre;

```

Curso	Total
Algebra	2
Biologia	1
Historia	1
Programacion	1
Fisica	1
Etica	0

7.4. Consulta 4: Promedio de notas por curso

```

1 SELECT c.nombre,
2 AVG(n.nota) AS promedio
3 FROM cursos c
4 JOIN inscripciones i ON i.id_curso = c.id
5 JOIN notas n ON n.id_inscripcion = i.id
6 GROUP BY c.id, c.nombre;

```

Curso	Promedio
Algebra	8.00
Biologia	9.00
Historia	6.80
Programacion	9.20
Fisica	7.00

7.5. Consulta 5: Estudiantes con notas mayores a 8.0

```

1 SELECT e.nombre, e.apellido, c.nombre, n.nota
2 FROM estudiantes e
3 JOIN inscripciones i ON i.id_estudiante = e.id
4 JOIN cursos c ON c.id = i.id_curso
5 JOIN notas n ON n.id_inscripcion = i.id
6 WHERE n.nota > 8.0;

```

Nombre	Apellido	Curso	Nota
Juan	Perez	Algebra	8.5
Juan	Perez	Biologia	9.0
Carlos	Sanchez	Programacion	9.2

7.6. Consulta 6: Cursos con mas de una inscripcion

```

1 SELECT c.nombre,
2 COUNT(i.id_estudiante) AS total_inscripciones
3 FROM cursos c
4 JOIN inscripciones i ON i.id_curso = c.id
5 GROUP BY c.id, c.nombre
6 HAVING COUNT(i.id_estudiante) > 1;

```

Curso	Inscripciones
Algebra	2

7.7. Consulta 7: Profesores del departamento Ingenieria

```

1 SELECT p.nombre, p.apellido
2 FROM profesores p
3 JOIN departamentos d ON d.id = p.id_departamento
4 WHERE d.nombre = 'Ingenieria';

```

Nombre	Apellido
Pedro	Martinez

7.8. Consulta 8: Cursos programados los lunes

```

1 SELECT c.nombre, h.dia_semana
2 FROM cursos c
3 JOIN horarios h ON h.id_curso = c.id
4 WHERE h.dia_semana = 'Lunes';

```

Curso	Dia
Algebra	Lunes

7.9. Consulta 9: Aulas con capacidad mayor a 25

```

1 SELECT nombre, capacidad
2 FROM aulas
3 WHERE capacidad > 25;

```

Aula	Capacidad
Aula 101	30
Sala Conferencias	50

7.10. Consulta 10: Cursos con aula y horario

```

1 SELECT c.nombre AS curso,
2        a.nombre AS aula,
3        h.dia_semana,
4        h.hora_inicio,
5        h.hora_fin
6 FROM cursos c
7 JOIN horarios h ON h.id_curso = c.id
8 JOIN aulas a ON a.id = h.id_aula;

```

Curso	Aula	Dia	Horario
Algebra	Aula 101	Lunes	08:00-10:00
Biologia	Aula 102	Martes	10:00-12:00
Historia	Sala Conferencias	Miercoles	13:00-15:00
Programacion	Laboratorio 1	Jueves	09:00-11:00
Fisica	Aula 102	Viernes	08:00-10:00

7.11. Consulta 11: Estudiante con la nota mas alta

```
1 SELECT e.nombre ,
2         e.apellido ,
3         c.nombre AS curso ,
4         n.nota
5 FROM estudiantes e
6 JOIN inscripciones i ON i.id_estudiante = e.id
7 JOIN cursos c ON c.id = i.id_curso
8 JOIN notas n ON n.id_inscripcion = i.id
9 WHERE n.nota = (
10        SELECT MAX(nota)
11        FROM notas
12 );
```

Nombre	Apellido	Curso	Nota
Carlos	Sanchez	Programacion	9.2

7.12. Consulta 12: Cantidad de profesores por departamento

```
1 SELECT d.nombre AS departamento ,
2        COUNT(p.id) AS total_profesores
3 FROM departamentos d
4 LEFT JOIN profesores p
5 ON p.id_departamento = d.id
6 GROUP BY d.id, d.nombre;
```

Departamento	Profesores
Matematicas	1
Ciencias	1
Humanidades	1
Ingenieria	1

7.13. Consulta 13: Cursos sin estudiantes inscritos

```
1 SELECT c.nombre
2 FROM cursos c
3 LEFT JOIN inscripciones i
4 ON i.id_curso = c.id
5 WHERE i.id IS NULL;
```

Curso
Etica

7.14. Consulta 14: Estudiantes inscritos en mas de un curso

```

1 SELECT e.nombre ,
2         e.apellido ,
3         COUNT(i.id_curso) AS total_cursos
4 FROM estudiantes e
5 JOIN inscripciones i
6 ON i.id_estudiante = e.id
7 GROUP BY e.id, e.nombre, e.apellido
8 HAVING COUNT(i.id_curso) > 1;

```

Nombre	Apellido	Total Cursos
Juan	Perez	2
Maria	Gomez	2

7.15. Consulta 15: Curso con menor promedio de notas

```

1 SELECT c.nombre ,
2         AVG(n.nota) AS promedio
3 FROM cursos c
4 JOIN inscripciones i ON i.id_curso = c.id
5 JOIN notas n ON n.id_inscripcion = i.id
6 GROUP BY c.id, c.nombre
7 ORDER BY promedio ASC
8 LIMIT 1;

```

Curso	Promedio
Historia	6.80